

Pseudocode:

```
function minSwapsCouples(row)
    n = length(row) / 2
    index_map = {person: index for index, person in enumerate(row)}
    swaps = 0

    for i = 0 to length(row) step 2
        first_person = row[i]
        second_person = first_person XOR 1

        if row[i + 1] != second_person then
            partner_index = index_map[second_person]
            swap row[i + 1] with row[partner_index]
            update index_map[row[partner_index]] = partner_index
            update index_map[row[i + 1]] = i + 1
            swaps += 1

    return swaps
```

Induction, Limit, and Step Counts:

❖ Induction:

- The loop works by going through the array in step 2. For each pair, it checks if the correct couple is sitting next to each other. If not, a swap will be performed, and the indices are updated in the `index_map`. The number of steps that occur will depend on the number of swaps needed for the couples to be together.

❖ Limit:

- For each pair, the worst-case scenario involves finding and swapping the partner, which takes constant time because of the use of our `index_map`. The algorithm processes each pair exactly once, meaning the maximum number of swaps will equal the number of pairs in the row.

❖ Step Counts:

- The loop runs from 0 to n (which is `length(row) / 2`), stepping by 2.

- The dictionary `index_map` lookup and update operations take constant time, i.e., $O(1)$. The total number of operations inside the loop, including checking the condition and performing a swap, is constant for each pair. Therefore, the loop runs $n/2$ times, with constant work per iteration.

Time Complexity:

- ❖ **Building the index map:** The map is created with a single pass over the input list, so it takes $O(n)$, where n is the length of the row.
- ❖ **Loop through the list:** The loop runs for each pair (half the size of the input), and within each loop, all operations are $O(1)$ due to the constant-time dictionary lookups and swaps. Therefore, the time complexity of the algorithm is $O(n)$, where n is the length of the input array row.